What a second board changed in the AArch32 port

The AArch32 port was written for one board — a Raspberry Pi Zero 2 W, four Cortex-A53 running in 32-bit mode — and then, in step 3, asked to carry a second: the **Luckfox Lyra B**, a Rockchip RK3506 with three Cortex-A7 and a Cortex-M0.

This document is the record of what that second board actually changed, and why. It is not a summary of the Lyra; it is a list of the places where one board had been mistaken for the architecture.

---

1. The shape of the repository

`BOARD` already existed, with two values that were the same silicon (`zero2w`, `rpi3b`). It now selects between two genuinely different SoCs:

```
micro-os-plus-iii-aarch32/
├── include/                  ARMv7-A, every board
│   ├── cmsis_device.h  exception_handler.hpp  hw_result.hpp  mmu.hpp
│   ├── semihosting.hpp
│   ├── port_ctx.hpp          NEW — the saved-context frame
│   ├── timer_arm.hpp         MOVED UP — CP15 generic-timer accessors
│   └── cmsis-plus/…          the port contract headers
├── src/                      ARMv7-A, every board
│   ├── context_switch.cpp  exception_handler.cpp  handlers.cpp
│   ├── smp_secondary.cpp
│   └── rtos/os-core.cpp      MOVED UP — the port's half of the scheduler
└── boards/
    ├── rpi-zero-2w/          BCM2837, 4× Cortex-A53
    ├── rpi3b/                BCM2837 too — shares rpi-zero-2w's src/ and
    │                         include/ under -DBOARD_RPI3B, owns its test/
    └── luckfox-lyra/         RK3506,  3× Cortex-A7
        ├── src/{mmu,port_sys,smp,timer_arm}.cpp  startup.S
        │   └── rtos/port_isr.cpp
        ├── include/{gic,led,osal,rk3506,smp,uart}.hpp
        ├── test/             this board's test applications
        ├── linker.ld        one map: this board runs on hardware
        └── openocd.cfg  hw.sh
```

The Cortex-M0 is not a fourth CPU and is not in this tree. It is a different ISA outside the A7 coherency domain: it cannot take an IPI as a peer and the scheduler cannot place a thread on it. The **board** is multi-

core; the **SMP cluster** is the three identical A7s. That is why the Lyra is a board directory rather than an architecture, and why `OS_NCPU` is 3.

The predecessor project stated the same fact by hard-coding `#define OS_NCPU 3` in `include/cmsis-plus/rtos/port/os-c-decls.h` — a board fact written into an architecture header. Here it is `NCPU 3` in the board's branch of `test/CMakeLists.txt`, and the header is shared unchanged.

---

2. Three files moved up, because one copy cannot drift

`src/rtos/os-core.cpp`

The port's half of the scheduler existed as two per-board copies, 575 and 486 lines. They were not two implementations of one design; they were the same file at two different ages, and the older one was missing fixes:

the shared file has	the older copy had
`vfp[32]` **and** `vfp_hi[32]` — d0–d31 saved	only d0–d15, so a thread preempted mid-NEON is silently corrupted
`reschedule()` defers when this core holds the kernel spinlock	switches anyway, stranding owner/depth on the outgoing stack
`switch_stacks()` clears owner/depth **before** the lock word	clears the lock first — a two-instruction window in which another core's critical section is wiped, and every core then spins forever
frame offsets derived with `offsetof(ctx_t, cpsr)`	the literals `34` and `200`
`lock_state` is `volatile`	not volatile
arrays sized `[OS_NCPU]`	sized `[3]`

Keeping both copies would have meant porting each of those fixes twice, in perpetuity, with nothing to notice when one was missed. There is one copy.

`include/timer_arm.hpp`

Every accessor in it is a CP15 instruction. That is a fact about ARMv7-A, not about a board.

`include/port_ctx.hpp` **(new)**

`ctx_t` and the offsets derived from it. It is shared because the frame is built by this port's own assembly (`handlers.cpp`, `context_switch.cpp`) and read by both the shared scheduler and each board's ISR, and all four have to agree.

---

3. What stayed per board, and the reason in each case

file	why it cannot be shared
`src/rtos/port_isr.cpp`	The BCM2837 has **no GIC**: the taken interrupt is read from a per-core local "IRQ source" register and the IPI is cleared through a mailbox. The RK3506 has a GIC-400: the interrupt comes from the CPU interface's IAR and every one of them needs an EOI with the same value. The dispatch is the interrupt controller.
`src/timer_arm.cpp`	The Pi has to **measure** its counter rate against the BCM2835 1 MHz system timer, because its `CNTFRQ` reports 19.2 MHz while the counter actually increments at ~1 MHz. The RK3506's `CNTFRQ` is programmed by the miniloader and is honest.
`src/smp.cpp`, `src/startup.S`	Releasing the secondaries: the Pi writes an entry address into the VideoCore armstub's mailbox-3 spin table (and `__smp_spin` + `SEV` under QEMU); the Lyra publishes an entry into an SRAM mailbox and clears the core's reset bits in the CRU.
`src/mmu.cpp`, `linker.ld`	Different memory maps — DRAM at `0x0001_0000` vs `0x0020_0000`, peripherals at `0x3F00_0000` vs `0xFF00_0000`.
`include/uart.hpp`	PL011 vs a DesignWare 16550.
`include/led.hpp`	A plain SoC GPIO vs GPIO1_A0 behind three CRU gates and a pinmux. On the Lyra this header also replaces thirteen verbatim copies of the same ungate/de-reset/mux sequence, one in each of the predecessor project's test directories.
`PORT_RAM_BASE` / `PORT_RAM_END` in `include/smp.hpp`	The DRAM window a stack pointer is checked against before the port switches to it.

---

4. Seven portability defects the second board exposed

These were all in code that had been shared for some time. None of them was visible while every board had four cores and the same drivers.

1. Every application compiled the SD and USB drivers. `uos_add_app()` linked `micro-os-plus::devices` whenever the target existed, so the `LIBRARIES` argument each test already passed was dead. On the Pi this was invisible. On the Lyra it is a hard failure — `usb_dwc2.cpp` includes `bcm2837.hpp` — which is how it was found. The caller now names what it links.

2. `wn[OS_NCPU] = {"w0", "w1", "w2", "w3"}` (`smp_test2`) — *too many initializers* at `OS_NCPU` 3. Names are generated now.

3. `port_cpu_id() & (OS_NCPU-1)` **and** `port_cpu_id() & 3u` — a modulo only while the core count is a power of two. At three cores, core 2 lands in slot 0 and the work-distribution report is wrong **without failing**. Replaced by `cpu_slot()`, which clamps.

4. Six tests named cores 1, 2 and 3 by hand to wait for the secondaries, print the join line and fold it into the verdict — reading one element past the end of `g_core_stage[OS_NCPU]`. Replaced by three shared helpers: `test_wait_secondaries()`, `test_secondaries_joined()`, `test_join_summary()`.

5. `g_core_produced[4]`, `g_core_consumed[4]`, `g_core_yields[4]` (`smp-pro-cons-test`) — fixed-size per-core tallies, printed as `c0..c3` and checked as `c0..c3`. Now `[OS_NCPU]`, printed and checked by loop (`report_per_core()`).

6. `boot_core3()` in the shared `src/smp_secondary.cpp` writes `g_core_stage[3]`. Guarded with `#if OS_NCPU > 3`.

7. `{0, 0, 0, 0}` **initialisers** for `[OS_NCPU]` arrays in `os-core.cpp`. Now `{}`.

Everything above is in the **shared** kernel and test tree, so the fixes apply to AArch64 as well. Both ports were re-run afterwards: 11 passed, 1 skipped (`usb_test`, no USB device model), 0 failed, on each.

---

5. Build-system changes

```
cmake -S . -B build-lyra -DBOARD=luckfox-lyra \
      -DCMAKE_TOOLCHAIN_FILE=../micro-os-plus-iii/cmake/toolchains/arm-none-eabi.cmake
```

knob	zero2w / rpi3b	luckfox-lyra
`BOARD`	`rpi-zero-2w`	`luckfox-lyra`
CPU flags	`-mcpu=cortex-a53 -mfpu=neon-fp-armv8`	`-mcpu=cortex-a7 -mfpu=neon-vfpv4`
SoC define	`SOC_BCM2837`	`SOC_RK3506`
extra library	`micro-os-plus::soc-bcm2837`	—
`UOS_BOARD_NCPU`	4	3
`hwd` linker script	`linker.ld`	`linker.ld`
`qemu` linker script	`linker.ld`	— none; hardware-only board (§6)
test applications	all twelve, plus the three harness suites	eleven of the twelve, plus eight of its own (and the harness's `mutex-stress-test`, built by the harness platform)

Two of those rows are new shapes, not just new values:

- **The linker script is now per variant, not only per board.** The Lyra's emulated image is linked for a different machine than its hardware image.

- `QEMU_BUILD` is now defined for the `qemu` variant. It does not mean "not hardware" — it means *this image runs under the emulator*, which for a board QEMU does not model is a different machine with a different console, a different interrupt controller and a different DRAM base.

Why eleven applications and not twelve

`sd_test`, `smp-mat-sdcard-test`, `smp-num-test`, `smp-pipeline-test` and `usb_test` link `micro-os-plus::devices`, whose backends were written against the BCM2837's EMMC and DWC2. The RK3506 has neither at those addresses, so when this chapter was first written the Lyra built only the seven that need nothing but CPUs, a timer and a console — which was also the only subset its predecessor project ever had.

Four of those five have since come back, and the count in the table above is the current one: eleven of the twelve shared applications, plus eight the Lyra owns — `smp_test5`, `smp_test6`, `smp_test7` and the five `smp_test_int*`, which drive the USB gadget, the C++ FatFs and the GIC-400 SGI paths no other board has. Nineteen images in all. Only `usb_test` is still missing, waiting on the RK3506 DWC2 device stack. What changed is below.

The selection is not configured at all any more — it is observed. Every board owns its tests in `test/<board>/`, and the port builds the directories that are there. A board with no SD card has no `sd_test` directory; a single-CPU board will have no `smp_*` ones. That is what makes the next port tractable: `cortexm` will have a dozen boards, half of them single-core, and none of them will appear in an `if/elseif` chain. The layout is `tests-in-aarch32-aarch64.md`.

Since this chapter was first written, the Lyra **has** gained its SD card: the predecessor's RK3506 DesignWare MSHC driver is now `soc/rk3506/` of the aarch32 port itself (it was in `micro-os-plus-iii-devices` until that repository was dissolved into the ports on 2026-10-06), reached through `micro-os-plus::devices-rk3506`, which the port's `CMakeLists.txt` defines, and the board builds 11 of the 12 shared applications plus eight of its own. Only `usb_test` is still missing, waiting on the RK3506 DWC2 device stack.

> `UOS_BOARD_CAPS` **is about the port, not the board.** It lists what *this port can drive on this board* — not the connectors the board carries. The Lyra B **does** have a microSD slot; it is where the miniloader lives. It does not declare `sdcard`, because `micro-os-plus::devices` has exactly one SD backend — `src/sd.cpp`, a polled Arasan SDHCI at the BCM2837's `0x3F30_0000` — and the RK3506 has a Synopsys DesignWare MSHC somewhere else. The capability returns the day a `dw_mmc` backend appears behind `sd.hpp`. Reading the list as a hardware inventory is the one way to misread it. (That backend has since arrived — see above.)

---

6. QEMU: the Lyra does not use it

The Pi runs under `-M raspi3b` through a 20-line AArch64 stub, because QEMU starts its Cortex-A53 cores in AArch64 and the port is 32-bit.

The Lyra has no emulated suite at all. This was considered, costed and closed; it is not unfinished work.

QEMU has no RK3506 machine, so the only candidate was `-M virt`, the generic Cortex-A7 — PL011 at `0x0900_0000`, GICv2 at `0x0800_0000`, DRAM at `0x4000_0000`. That machine models none of the RK3506's own blocks: not the CRU, not the GIC-400 at `0xFF58_0000`, not the DesignWare SD host, not the DWC2 gadget, not the SRAM mailbox, not the Cortex-M0. An emulated Lyra would therefore be a generic Cortex-A7 wearing the board's name, running the subset of tests that are not about this board — while the DWC2 gadget, the SD host and the M0 mailbox, which are the reason the board is in this project, would be exactly the parts not modelled.

Nothing about the *invocation* was the obstacle, and it is worth writing down so nobody re-derives it: `qemu-system-arm -M virt -cpu cortex-a7 -smp 3` is accepted and starts (verified on xPack QEMU 9.2.4). What stopped it was inside the image. `linker-qemu-virt.ld` placed the `-qemu` build at `0x4000_0000`, and then `src/mmu.cpp` built a page table that maps DRAM below `0x0800_0000` and MMIO at or above `0xFF00_0000`, with **everything else faulting** — so the image declared its own code, its own stack, virt's PL011 and virt's GIC to be invalid addresses, and died the instant `mmu_enable()` set SCTLR.M. Two `#if defined(QEMU_BUILD)` branches would have fixed it, in `gic.hpp` and `mmu.cpp`, the same way `uart.hpp`, `led.hpp` and `smp.hpp` already have theirs. The fix was small; the thing it bought was not worth having.

How the board says so

`test/boards/luckfox-lyra/board.cmake` sets **no** `UOS_BOARD_LINKER_QEMU`. That silence is the declaration — there is no second flag to keep in step with it and no way to claim an emulated target without having one. `test/CMakeLists.txt` reads it and builds one image per test instead of two, so the board went from 38 targets to 19. `linker-qemu-virt.ld` and the board's `qemu.sh` are gone, and `test/qemu.sh` answers `BOARD=luckfox-lyra` by naming the hardware runner:

```
$ BOARD=luckfox-lyra test/qemu.sh
qemu.sh: luckfox-lyra is tested on hardware -- it has no emulated suite.
         BOARD=luckfox-lyra test/hw.sh <test>
boards with an emulated suite:
  rpi3b
  rpi-zero-2w
```

The `QEMU_BUILD` branches still in `uart.hpp`, `led.hpp` and `smp.hpp` are inert — nothing defines the macro for this board. They were left alone rather than stripped, because the hardware path is the `#else` of each and those three files pass on hardware today.

---

7. Hardware: what the shared runner had to learn

`test_smpl/run-hw.sh` in the kernel repository drives every hardware session. It had three things hard-coded that were the Pi's, not the architecture's:

was	is
target names spelled `bcm2837.cpu$core`	`UOS_HW_TARGET_FMT`, a printf format — `rk3506.a7.%d` for the Lyra
every core 0..NCPU-1 halted, loaded and resumed	`UOS_HW_CORES`, the indices that are debug targets *at load time*
halt → load → resume, nothing in between	`UOS_HW_PRELOAD`, Tcl run after the halt and before `load_image`

and it gained a third resume mode, `entry` — resume at the entry point touching no register — beside `pc` (AArch64) and `cpsr` (AArch32 on the Pi).

Each of the three is a real property of the RK3506, not a convenience:

- **Only core 0 is examinable when the image is loaded.** Cores 1 and 2 are in the BootROM until the kernel clears their CRU reset bits, which is why `openocd.cfg` declares them `-defer-examine`. Halting a target that has not been examined fails.
- **There is no `__smp_spin`.** The Pi's secondaries park in a spin table the runner zeroes after the load so a core that reaches its parking loop before core 0 clears `.bss` cannot jump to a stale entry. The Lyra's are released through an SRAM mailbox instead, so `UOS_HW_SPIN_WORDS=0` and the stage is skipped entirely.
- **The MMU and caches are ON** when the miniloader hands core 0 over. `load_image` writing through a dirty cache leaves DRAM holding something other than the image, so `SCTLR.{M,C,I}` are cleared and the I-cache, branch predictor and TLB invalidated first — the same sequence the predecessor's own `write_board.sh` used. It is now `UOS_HW_PRELOAD` in `test/boards/luckfox-lyra/hw.sh`.

The Pi's generated script is unchanged by all of this, byte for byte apart from `[format {bcm2837.cpu%d} $core]` where it said `bcm2837.cpu$core`.

`test/boards/luckfox-lyra/openocd.cfg` is one copy; the predecessor had thirteen, one per test directory. `write_board.sh` is gone — `test/boards/luckfox-lyra/hw.sh` is what it did, for any test, and every board now has the same two scripts.

```
BOARD=luckfox-lyra test/hw.sh list
BOARD=luckfox-lyra test/hw.sh smp_test0
```

---

8. What is not done

- **Running** the RK3506-specific tests. All eight the predecessor had are now here and build — `smp_test_int` and `smp_test_int2` came over first, and `smp_test5`, `smp_test6`, `smp_test7`, `smp_test_int3`, `smp_test_int4` and `smp_test_int5` followed with the code they need (§9). They exercise GIC priority and preemption, the Rockchip USB gadget, the SD host and the Cortex-M0 mailbox, and every one of them is hardware-only by nature. None has been run.

`smp_test0`–`smp_test4` exist in both projects under the same names but are **different tests**; the unified ones are in use.

- `smp_test4` on the Lyra is reported not to work. The symptom has not been captured, and the two candidate causes — the balancer leaving a core idle, versus the test dying earlier — are different bugs.
- A per-test linker override. `smp-pro-cons-test` carries its own linker script in the predecessor, and the build has no mechanism for one.

Emulation is **not** on this list. See §6. - Hardware validation on a Lyra. Nothing in this document claims a Lyra has run.